

```
pip install anndata scanpy pooch scipy matplotlib pandas
```

```
Requirement already satisfied: pynndescent>=0.5.13 in /usr/local/lib/python3.1  
Requirement already satisfied: scikit-learn>=1.6 in /usr/local/lib/python3.12/  
Requirement already satisfied: seaborn>=0.13.2 in /usr/local/lib/python3.12/di  
Collecting session-info2 (from scanpy)
```

```
  Downloading session_info2-0.4.1-py3-none-any.whl.metadata (2.5 kB)
```

```
Requirement already satisfied: statsmodels>=0.14.5 in /usr/local/lib/python3.1  
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages  
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/  
Requirement already satisfied: umap-learn>=0.5.12 in /usr/local/lib/python3.12  
Requirement already satisfied: platformdirs>=2.5.0 in /usr/local/lib/python3.1  
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.12/d  
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/d  
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/  
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/  
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-pac  
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/d  
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.  
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dis  
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/py  
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-pack
```

Uninstalling pandas-2.2.2:

Successfully uninstalled pandas-2.2.2

ERROR: pip's dependency resolver does not currently take into account all the google-colab 1.0.0 requires pandas==2.2.2, but you have pandas 2.3.3 which is Successfully installed anndata-0.12.14 array-api-compat-1.14.0 donfig-0.8.1.po

```
import numpy as np
import pandas as pd
import anndata as ad
import anndata
import matplotlib.pyplot as plt
import scanpy as sc
import pooch
from scipy.sparse import csr_matrix
```

```
print("anndata version:", ad.__version__)
```

```
anndata version: 0.12.14
```

```
/tmp/ipykernel_31334/2060735975.py:10: FutureWarning: `__version__` is deprecated
```

```
print("anndata version:", ad.__version__)
```

```
counts = csr_matrix(np.random.poisson(1, size=(100, 2000)), dtype=np.float32)
adata = ad.AnnData(counts)
print("Initial AnnData:\n", adata)
print("adata.X (sparse matrix):\n", adata.X)
```

Initial AnnData:

AnnData object with n_obs × n_vars = 100 × 2000

adata.X (sparse matrix):

<Compressed Sparse Row sparse matrix of dtype 'float32'
with 126497 stored elements and shape (100, 2000)>

Coords	Values
(0, 0)	2.0
(0, 1)	1.0
(0, 5)	1.0
(0, 7)	3.0
(0, 8)	2.0
(0, 10)	2.0
(0, 16)	1.0
(0, 17)	1.0
(0, 18)	1.0
(0, 19)	2.0
(0, 20)	2.0
(0, 21)	2.0
(0, 24)	1.0
(0, 25)	1.0
(0, 26)	1.0
(0, 27)	2.0
(0, 29)	1.0
(0, 30)	1.0
(0, 31)	1.0
(0, 32)	1.0

```

(0, 34)      1.0
(0, 36)      1.0
(0, 37)      1.0
(0, 38)      1.0
(0, 39)      1.0
:           :
(99, 1959)   1.0
(99, 1963)   1.0
(99, 1965)   2.0
(99, 1966)   1.0
(99, 1967)   1.0
(99, 1968)   4.0
(99, 1971)   1.0
(99, 1972)   3.0
(99, 1973)   1.0
(99, 1975)   1.0
(99, 1976)   1.0
(99, 1977)   1.0
(99, 1978)   1.0
(99, 1979)   1.0
(99, 1980)   2.0
(99, 1983)   2.0
(99, 1984)   1.0
(99, 1985)   1.0
(99, 1988)   1.0
(99, 1989)   1.0
(99, 1993)   1.0
(99, 1996)   1.0
(99, 1997)   3.0
(99, 1998)   1.0
(99, 1999)   3.0

```

```

adata.obs_names = [f"Cell_{i:d}" for i in range(adata.n_obs)]
adata.var_names = [f"Gene_{i:d}" for i in range(adata.n_vars)]
print("First 10 obs_names:", adata.obs_names[:10].tolist())

```

```

First 10 obs_names: ['Cell_0', 'Cell_1', 'Cell_2', 'Cell_3', 'Cell_4', 'Cell_5',

```

```

subset_view = adata[["Cell_1", "Cell_10"], ["Gene_5", "Gene_1900"]]
print("Subset view:", subset_view)

```

```

Subset view: View of AnnData object with n_obs × n_vars = 2 × 2

```

```

ct = np.random.choice(["B", "T", "Monocyte"], size=(adata.n_obs,))
adata.obs["cell_type"] = pd.Categorical(ct)
print("adata.obs (first 5 rows):\n", adata.obs.head())
print("Updated AnnData:", adata)

```

```

adata.obs (first 5 rows):
      cell_type
Cell_0         B
Cell_1         B

```

```
Cell_2      T
Cell_3      T
Cell_4      B
Updated AnnData: AnnData object with n_obs × n_vars = 100 × 2000
  obs: 'cell_type'
```

```
bdata = adata[adata.obs.cell_type == "B"]
print("B-cell subset:", bdata)
```

```
B-cell subset: View of AnnData object with n_obs × n_vars = 30 × 2000
  obs: 'cell_type'
```

```
adata.obsm["X_umap"] = np.random.normal(0, 1, size=(adata.n_obs, 2))
adata.varm["gene_stuff"] = np.random.normal(0, 1, size=(adata.n_vars, 5))
print("obsm keys:", list(adata.obsm.keys()))
print("AnnData after obsm/varm:", adata)
```

```
obsm keys: ['X_umap']
AnnData after obsm/varm: AnnData object with n_obs × n_vars = 100 × 2000
  obs: 'cell_type'
  obsm: 'X_umap'
  varm: 'gene_stuff'
```

```
adata.uns["random"] = [1, 2, 3]
print("adata.uns:", dict(adata.uns))
```

```
adata.uns: {'random': [1, 2, 3]}
```

```
adata.layers["log_transformed"] = np.log1p(adata.X)
print("AnnData with layers:", adata)
```

```
AnnData with layers: AnnData object with n_obs × n_vars = 100 × 2000
  obs: 'cell_type'
  uns: 'random'
  obsm: 'X_umap'
  varm: 'gene_stuff'
  layers: 'log_transformed'
```

```
df_log = adata.to_df(layer="log_transformed")
print("log_transformed DataFrame (first 3 rows, first 5 cols):\n", df_log.iloc[
```

```
log_transformed DataFrame (first 3 rows, first 5 cols):
      Gene_0  Gene_1  Gene_2  Gene_3  Gene_4
Cell_0  1.098612  0.693147  0.000000  0.000000  0.000000
Cell_1  0.693147  0.693147  1.098612  0.000000  0.693147
Cell_2  1.098612  0.000000  0.693147  0.693147  0.000000
```

```
adata.write("my_results.h5ad", compression="gzip")
print("Saved to my_results.h5ad")
```

Saved to my_results.h5ad

```
obs_meta = pd.DataFrame(
    {
        "time_yr":      np.random.choice([0, 2, 4, 8], adata.n_obs),
        "subject_id":   np.random.choice(["subject 1", "subject 2", "subject
        "instrument_type": np.random.choice(["type a", "type b"], adata.n_obs),
        "site":         np.random.choice(["site x", "site y"], adata.n_obs),
    },
    index=adata.obs.index,
)
adata = ad.AnnData(adata.X, obs=obs_meta, var=adata.var)
print("AnnData with rich obs metadata:", adata)

# View
view_5x2 = adata[:5, ["Gene_1", "Gene_3"]]
print("View (5x2):", view_5x2)

# Explicit copy
adata_subset = adata[:5, ["Gene_1", "Gene_3"]].copy()
print("Copy from view:", adata_subset)

# Modify through a view
print("Before:", adata[:3, "Gene_1"].X.toarray().tolist())
adata[:3, "Gene_1"].X = [0, 0, 0]
print("After zeroing:", adata[:3, "Gene_1"].X.toarray().tolist())

# Implicit copy via attribute assignment (triggers ImplicitModificationWarning)
adata_sub3 = adata[:3, ["Gene_1", "Gene_2"]]
adata_sub3.obs["foo"] = range(3)
print("After implicit copy:", adata_sub3)

# Boolean indexing
print("Cells from time_yr 2 or 4 (first 5):\n", adata[adata.obs.time_yr.isin([2
```

```
AnnData with rich obs metadata: AnnData object with n_obs x n_vars = 100 x 2000
  obs: 'time_yr', 'subject_id', 'instrument_type', 'site'
View (5x2): View of AnnData object with n_obs x n_vars = 5 x 2
  obs: 'time_yr', 'subject_id', 'instrument_type', 'site'
Copy from view: AnnData object with n_obs x n_vars = 5 x 2
  obs: 'time_yr', 'subject_id', 'instrument_type', 'site'
Before: [[1.0], [1.0], [0.0]]
After zeroing: [[0.0], [0.0], [0.0]]
After implicit copy: AnnData object with n_obs x n_vars = 3 x 2
  obs: 'time_yr', 'subject_id', 'instrument_type', 'site', 'foo'
Cells from time_yr 2 or 4 (first 5):
      time_yr subject_id instrument_type  site
Cell_0      4  subject 1           type a  site x
Cell_1      4  subject 4           type a  site x
Cell_9      4  subject 2           type b  site x
Cell_10     4  subject 1           type b  site y
Cell_11     4  subject 1           type b  site x
```

```

/usr/local/lib/python3.12/dist-packages/anndata/_core/anndata.py:639: FutureWarn
if self._handle_view_X_cow(value):
/tmp/ipykernel_31334/3198331881.py:23: ImplicitModificationWarning: Modifying `X
adata[:3, "Gene_1"].X = [0, 0, 0]
/tmp/ipykernel_31334/3198331881.py:28: ImplicitModificationWarning: Trying to mo
adata_sub3.obs["foo"] = range(3)

```

```

# OLD (deprecated/removed):
# adata_backed = ad.read("my_results.h5ad", backed="r")

# NEW – use read_h5ad directly:
adata_backed = ad.io.read_h5ad("my_results.h5ad", backed="r")
print("Is backed:", adata_backed.isbacked)
print("Backed filename:", adata_backed.filename)
adata_backed.file.close()

```

```

Is backed: True
Backed filename: my_results.h5ad

```

```

# 1.12 Backed / partial reading
adata_backed = ad.io.read_h5ad("my_results.h5ad", backed="r")
print("Is backed:", adata_backed.isbacked)
print("Backed filename:", adata_backed.filename)
adata_backed.file.close()

```

```

Is backed: True
Backed filename: my_results.h5ad

```

```

datapath = pooch.retrieve(
    path=pooch.os_cache("scverse_tutorials"),
    url=(
        "https://exampledata.scverse.org/tutorials/"
        "scverse-getting-started-anndata-pbmc3k_processed.h5ad"
    ),
    known_hash="md5:b80deb0997f96b45d06f19c694e46243",
)
adata = anndata.read_h5ad(datapath)
print("Loaded AnnData:\n", adata)

```

Downloading data from 'https://exampledata.scverse.org/tutorials/scverse-getting-started-anndata-pbmc3k_processed.h5ad'
Loaded AnnData:

```

AnnData object with n_obs × n_vars = 2638 × 11505
  obs: 'n_genes', 'percent_mito', 'n_counts', 'louvain_cell_types'
  var: 'gene_names', 'n_cells', 'gene_ids'
  uns: 'louvain', 'louvain_colors', 'pca'
  obsm: 'X_pca', 'X_tsne', 'X_umap'
  layers: 'raw'
  obsp: 'distances_all'

```

```
data.X (sparse):\n", adata.X)
```

```

on-zero values sample:", adata.X.data[:5])
on-zero indices sample:", adata.X.indices[:5])
parsity (fraction non-zero):", round(adata.X.nnz / np.prod(adata.X.shape), 4))

```

```

adata.X (sparse):
<Compressed Sparse Row sparse matrix of dtype 'float32'
  with 2076576 stored elements and shape (2638, 11505)>
  Coords      Values
(0, 19)      0.6496620774269104
(0, 52)      0.6496620774269104
(0, 58)      1.0402015447616577
(0, 118)     0.6496620774269104
(0, 131)     0.6496620774269104
(0, 146)     0.6496620774269104
(0, 147)     0.6496620774269104
(0, 180)     3.6509344577789307
(0, 181)     0.6496620774269104
(0, 182)     0.6496620774269104
(0, 208)     1.0402015447616577
(0, 210)     1.7181968688964844
(0, 263)     0.6496620774269104
(0, 270)     0.6496620774269104
(0, 291)     0.6496620774269104
(0, 313)     0.6496620774269104
(0, 324)     0.6496620774269104
(0, 336)     0.6496620774269104
(0, 357)     1.0402015447616577
(0, 362)     0.6496620774269104
(0, 380)     2.6894383430480957
(0, 394)     0.6496620774269104
(0, 402)     0.6496620774269104
(0, 411)     0.6496620774269104
(0, 418)     0.6496620774269104
:           :
(2637, 11103) 0.7506172060966492
(2637, 11191) 0.7506172060966492
(2637, 11210) 0.7506172060966492
(2637, 11221) 0.7506172060966492
(2637, 11245) 1.1745277643203735
(2637, 11257) 1.1745277643203735
(2637, 11284) 2.996293067932129
(2637, 11296) 0.7506172060966492
(2637, 11298) 1.1745277643203735
(2637, 11304) 0.7506172060966492
(2637, 11309) 0.7506172060966492
(2637, 11310) 0.7506172060966492
(2637, 11319) 0.7506172060966492
(2637, 11357) 0.7506172060966492
(2637, 11361) 0.7506172060966492
(2637, 11416) 0.7506172060966492
(2637, 11443) 0.7506172060966492
(2637, 11455) 0.7506172060966492
(2637, 11470) 0.7506172060966492
(2637, 11492) 1.4713064432144165
(2637, 11493) 0.7506172060966492
(2637, 11494) 2.177945375442505

```

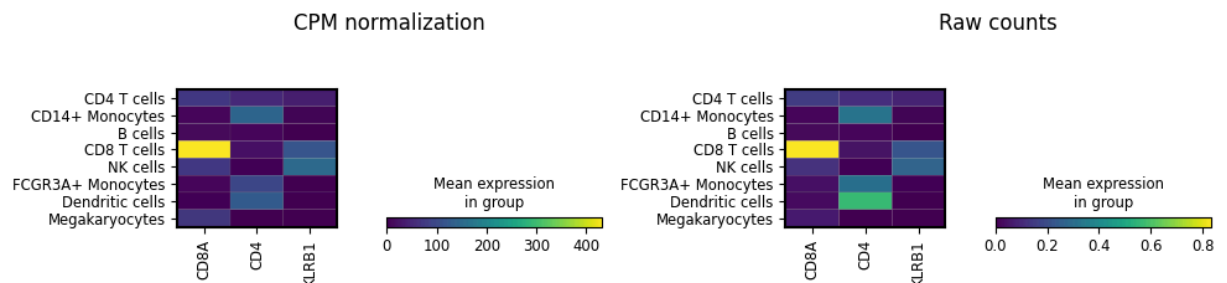
```
(2637, 11498) 0.7506172060966492
(2637, 11501) 0.7506172060966492
(2637, 11504) 1.4713064432144165
Non-zero values sample: [0.6496621 0.6496621 1.0402015 0.6496621 0.6496621]
Non-zero indices sample: [ 19  52  58 118 131]
Sparsity (fraction non-zero): 0.0684
```

```
print("Layers:", list(adata.layers.keys()))
print("raw layer dtype:", adata.layers["raw"].dtype)
print("raw layer sample values:", adata.layers["raw"].data[:5])

# Add counts-per-million layer
adata.layers["counts_per_million"] = adata.layers["raw"].copy()
sc.pp.normalize_total(adata, target_sum=10**6, layer="counts_per_million")
print("Layers after CPM:", list(adata.layers.keys()))

# Plot CPM vs raw
genes_of_interest = ["CD8A", "CD4", "KLRB1"]
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 2.5))
sc.pl.matrixplot(adata, groupby="louvain_cell_types", var_names=genes_of_interest,
                 layer="counts_per_million", ax=ax1, show=False)
ax1.set_title("CPM normalization")
sc.pl.matrixplot(adata, groupby="louvain_cell_types", var_names=genes_of_interest,
                 layer="raw", ax=ax2, show=False)
ax2.set_title("Raw counts")
plt.tight_layout()
plt.show()
```

```
Layers: ['raw']
raw layer dtype: int64
raw layer sample values: [1 1 2 1 1]
Layers after CPM: ['raw', 'counts_per_million']
```



```
print("obs columns:", adata.obs.keys().tolist())
print("obs head:\n", adata.obs.head())

# Count B cells
n_b = sum(adata.obs["louvain_cell_types"] == "B cells")
print(f"Number of B cells: {n_b}")

# Add quality flag
adata.obs["is_low_quality"] = adata.obs["percent_mito"] > 0.03
print("obs after quality flag:\n", adata.obs.head())
print("var head:\n", adata.var.head())
```



```
obs columns: ['n_genes', 'percent_mito', 'n_counts', 'louvain_cell_types']
```

```
obs head:
```

	n_genes	percent_mito	n_counts	louvain_cell_types
cell_barcode				
AAACATACAACCAC-1	781	0.030178	2419.0	CD4 T cells
AAACATTGAGCTAC-1	1352	0.037936	4903.0	B cells
AAACATTGATCAGC-1	1131	0.008897	3147.0	CD4 T cells
AAACCGTGCTTCCG-1	960	0.017431	2639.0	CD14+ Monocytes
AAACCGTGTATGCG-1	522	0.012245	980.0	NK cells

```
Number of B cells: 342
```

```
obs after quality flag:
```

	n_genes	percent_mito	n_counts	louvain_cell_types	\
cell_barcode					
AAACATACAACCAC-1	781	0.030178	2419.0	CD4 T cells	
AAACATTGAGCTAC-1	1352	0.037936	4903.0	B cells	
AAACATTGATCAGC-1	1131	0.008897	3147.0	CD4 T cells	
AAACCGTGCTTCCG-1	960	0.017431	2639.0	CD14+ Monocytes	
AAACCGTGTATGCG-1	522	0.012245	980.0	NK cells	

```
is_low_quality
```

cell_barcode	is_low_quality
AAACATACAACCAC-1	True
AAACATTGAGCTAC-1	True
AAACATTGATCAGC-1	False
AAACCGTGCTTCCG-1	False
AAACCGTGTATGCG-1	False

```
var head:
```

	gene_names	n_cells	gene_ids
gene_names			
LINC00115	LINC00115	18	ENSG00000225880
NOC2L	NOC2L	258	ENSG00000188976
KLHL17	KLHL17	9	ENSG00000187961
PLEKHN1	PLEKHN1	7	ENSG00000187583
HES4	HES4	145	ENSG00000188290

```
print("obs_names (first 5):", adata.obs_names[:5].tolist())
```

```
print("var_names (first 5):", adata.var_names[:5].tolist())
```

```
# Switch to Ensembl IDs
```

```
adata.var_names = adata.var["gene_ids"]
```

```
print("var after re-index to Ensembl:\n", adata.var.head(3))
```

```
# Restore gene names
```

```
adata.var_names = adata.var["gene_names"]
```

```
print("var restored:\n", adata.var.head(3))
```

```
obs_names (first 5): ['AAACATACAACCAC-1', 'AAACATTGAGCTAC-1', 'AAACATTGATCAGC-1']
```

```
var_names (first 5): ['LINC00115', 'NOC2L', 'KLHL17', 'PLEKHN1', 'HES4']
```

```
var after re-index to Ensembl:
```

	gene_names	n_cells	gene_ids
gene_ids			
ENSG00000225880	LINC00115	18	ENSG00000225880
ENSG00000188976	NOC2L	258	ENSG00000188976

```

ENSG00000187961    KLHL17          9  ENSG00000187961
var restored:
      gene_names  n_cells      gene_ids
gene_names
LINC00115  LINC00115      18  ENSG00000225880
NOC2L      NOC2L      258  ENSG00000188976
KLHL17      KLHL17      9   ENSG00000187961

```

```

# By name
adata_small = adata[:5, ["LYZ", "FOS", "MALAT1"]]
print("adata_small shape:", adata_small.shape)
print("X (dense):\n", adata_small.X.toarray())
print("raw layer:\n", adata_small.layers["raw"].toarray())
print("obs:\n", adata_small.obs)
print("var:\n", adata_small.var)

```

```

# Boolean: keep high-quality cells only
adata_hq = adata[~adata.obs["is_low_quality"], :]
print("High-quality cells:", adata_hq.n_obs)
print("adata_hq obs (first 5):\n", adata_hq.obs.head())

```

```

adata_small shape: (5, 3)
X (dense):
[[0.6496621  2.4036813  3.8249342 ]
 [0.8553989  0.64266497 4.174533  ]
 [0.878057   2.8014445  4.7918878 ]
 [3.0494576  1.6464083  2.3237872 ]
 [0.         0.         3.923651  ]]
raw layer:
[[ 1  11  49]
 [ 3   2 142]
 [ 2  22 170]
 [24   5  11]
 [ 0   0  22]]
obs:
      n_genes  percent_mito  n_counts  louvain_cell_types  \
cell_barcode
AAACATACAACCAC-1      781      0.030178    2419.0      CD4 T cells
AAACATTGAGCTAC-1     1352      0.037936    4903.0      B cells
AAACATTGATCAGC-1     1131      0.008897    3147.0      CD4 T cells
AAACCGTGCTTCCG-1      960      0.017431    2639.0  CD14+ Monocytes
AAACCGTGTATGCG-1      522      0.012245     980.0      NK cells

      is_low_quality
cell_barcode
AAACATACAACCAC-1      True
AAACATTGAGCTAC-1      True
AAACATTGATCAGC-1     False
AAACCGTGCTTCCG-1     False
AAACCGTGTATGCG-1     False
var:
      gene_names  n_cells      gene_ids
gene_names

```

```

LYZ          LYZ      1631  ENSG00000090382
FOS          FOS      2473  ENSG00000170345
MALAT1      MALAT1    2699  ENSG00000251562
High-quality cells: 2257
adata_hq obs (first 5):

```

	n_genes	percent_mito	n_counts	louvain_cell_types	\
cell_barcode					
AAACATTGATCAGC-1	1131	0.008897	3147.0	CD4 T cells	
AAACCGTGCTTCCG-1	960	0.017431	2639.0	CD14+ Monocytes	
AAACCGTGTATGCG-1	522	0.012245	980.0	NK cells	
AAACGCACTGGTAC-1	782	0.016644	2163.0	CD8 T cells	
AAACGCTGTAGCCA-1	533	0.011765	1275.0	CD4 T cells	

```


```

	is_low_quality
cell_barcode	
AAACATTGATCAGC-1	False
AAACCGTGCTTCCG-1	False
AAACCGTGTATGCG-1	False
AAACGCACTGGTAC-1	False
AAACGCTGTAGCCA-1	False

```

print("obs keys:", list(adata.obsm.keys()))
for key in adata.obsm:
    print(f" {key}: shape {adata.obsm[key].shape}")

# Plot PCA / t-SNE / UMAP
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
b_flag = adata.obs["louvain_cell_types"] == "B cells"
scatter_kw = dict(s=3, linewidth=0, cmap="coolwarm")

for ax, key, title in zip(axes, ["X_pca", "X_tsne", "X_umap"], ["PCA", "t-SNE",
    ax.scatter(adata.obsm[key][:, 0], adata.obsm[key][:, 1],
                c=b_flag.astype(int), **scatter_kw)
    ax.set_title(title)
    ax.axis("off")
    ax.set_aspect("equal")

plt.tight_layout()
plt.show()

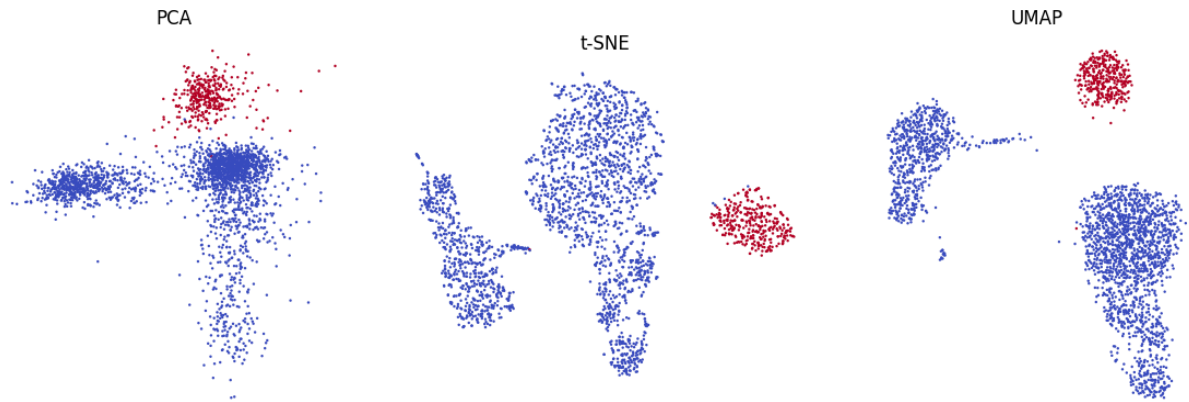
```

```
obsm keys: ['X_pca', 'X_tsne', 'X_umap']
```

```
X_pca: shape (2638, 50)
```

```
X_tsne: shape (2638, 2)
```

```
X_umap: shape (2638, 2)
```



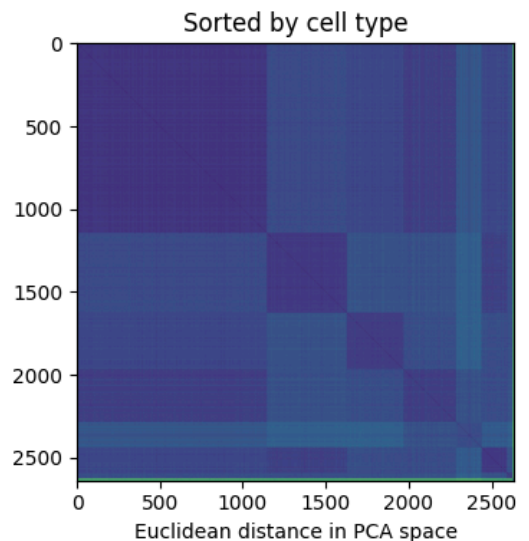
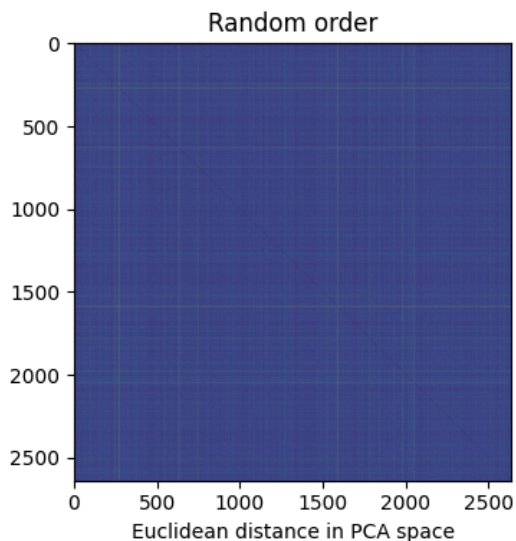
```
print("obsp keys:", list(adata.obsp.keys()))
print("distances_all shape:", adata.obsp["distances_all"].shape)
```

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].imshow(adata.obsp["distances_all"])
axes[0].set_title("Random order")
axes[0].set_xlabel("Euclidean distance in PCA space")
```

```
reorder = np.argsort(adata.obs["louvain_cell_types"])
axes[1].imshow(adata[reorder, :].obsp["distances_all"])
axes[1].set_title("Sorted by cell type")
axes[1].set_xlabel("Euclidean distance in PCA space")
```

```
plt.tight_layout()
plt.show()
```

```
obsp keys: ['distances_all']
distances_all shape: (2638, 2638)
```



```
print("uns keys:", list(adata.uns.keys()))
print("louvain params:", adata.uns["louvain"])
print("louvain colors:", adata.uns["louvain_colors"])
print("PCA variance (first 5):", adata.uns["pca"]["variance"][:5])
```

```
uns keys: ['louvain', 'louvain_colors', 'pca']
louvain params: {'params': {'random_state': array([0]), 'resolution': array([1])}
louvain colors: ['#1f77b4' '#ff7f0e' '#2ca02c' '#d62728' '#9467bd' '#8c564b' '#e
'#bcbd22']
PCA variance (first 5): [32.11044  18.718647 15.607319 13.235274  4.8012376]
```

```
# Create a view
adata_view = adata[:5, 5:10]
print("View:", adata_view)

# Parent change propagates to view
print("Before modification:\n", adata_view.X.toarray())
adata.X[0, 7] = 99
print("After modifying parent adata.X[0,7]=99:\n", adata_view.X.toarray())

# Explicit copy
adata_view_copy = adata_view.copy()
print("Explicit copy:", adata_view_copy)

# Implicit copy via obs assignment (triggers ImplicitModificationWarning)
adata_view.obs["new_column"] = "Test"
print("After obs assignment (now a real AnnData):", adata_view)

# Verify isolation from parent
adata.X[0, 8] = 98
print("Parent X[0,5:10]:\n", adata.X[:5, 5:10].toarray())
print("Former view X:\n", adata_view.X.toarray())
print("Explicit copy X:\n", adata_view_copy.X.toarray())
```

```
View: View of AnnData object with n_obs x n_vars = 5 x 5
  obs: 'n_genes', 'percent_mito', 'n_counts', 'louvain_cell_types', 'is_low_
  var: 'gene_names', 'n_cells', 'gene_ids'
  uns: 'louvain', 'louvain_colors', 'pca'
  obsm: 'X_pca', 'X_tsne', 'X_umap'
  layers: 'raw', 'counts_per_million'
  obsp: 'distances_all'
```

Before modification:

```
[[0.      0.      0.      0.      0.      ]
 [0.      0.      0.      0.64266497 0.      ]
 [0.532456 0.      0.      0.      0.      ]
 [2.1446393 0.      0.      0.      0.      ]
 [0.      0.      0.      0.      0.      ]]
```

After modifying parent adata.X[0,7]=99:

```
[[ 0.      0.      99.      0.      0.      ]
 [ 0.      0.      0.      0.64266497 0.      ]
 [ 0.532456 0.      0.      0.      0.      ]]
```

```

[ 2.1446393  0.          0.          0.          0.          ]
[ 0.          0.          0.          0.          0.          ]]
Explicit copy: AnnData object with n_obs × n_vars = 5 × 5
  obs: 'n_genes', 'percent_mito', 'n_counts', 'louvain_cell_types', 'is_low_
  var: 'gene_names', 'n_cells', 'gene_ids'
  uns: 'louvain', 'louvain_colors', 'pca'
  obsm: 'X_pca', 'X_tsne', 'X_umap'
  layers: 'raw', 'counts_per_million'
  obsp: 'distances_all'
After obs assignment (now a real AnnData): AnnData object with n_obs × n_vars
  obs: 'n_genes', 'percent_mito', 'n_counts', 'louvain_cell_types', 'is_low_
  var: 'gene_names', 'n_cells', 'gene_ids'
  uns: 'louvain', 'louvain_colors', 'pca'
  obsm: 'X_pca', 'X_tsne', 'X_umap'
  layers: 'raw', 'counts_per_million'
  obsp: 'distances_all'
Parent X[0,5:10]:
[[ 0.          0.          99.          98.          0.          ]
 [ 0.          0.          0.          0.64266497  0.          ]
 [ 0.532456    0.          0.          0.          0.          ]
 [ 2.1446393   0.          0.          0.          0.          ]
 [ 0.          0.          0.          0.          0.          ]]
Former view X:
[[ 0.          0.          99.          0.          0.          ]
 [ 0.          0.          0.          0.64266497  0.          ]
 [ 0.532456    0.          0.          0.          0.          ]
 [ 2.1446393   0.          0.          0.          0.          ]
 [ 0.          0.          0.          0.          0.          ]]
Explicit copy X:
[[ 0.          0.          99.          0.          0.          ]
 [ 0.          0.          0.          0.64266497  0.          ]
 [ 0.532456    0.          0.          0.          0.          ]
 [ 2.1446393   0.          0.          0.          0.          ]
 [ 0.          0.          0.          0.          0.          ]]
/usr/local/lib/python3.12/dist-packages/scipy/sparse/_index.py:168: SparseEffi
self._set_intXint(row, col, x.flat[0])
/tmp/ipykernel_31334/2990997188.py:15: ImplicitModificationWarning: Trying to
adata_view.obs["new_column"] = "Test"
/usr/local/lib/python3.12/dist-packages/scipy/sparse/_index.py:168: SparseEffi

```

Start coding or [generate](#) with AI.